

Logbook

RTOS Q4
(Linux Device Driver)

Contents

1. Before we start	3
2. Trying out the sample	4
2.1. The sample driver from Apriorit	4
2.1.1. <code>pr_notice</code> fun fact :	6
2.2. The Conclutions with Apriorit	10
3. Making my own device driver.	12
3.1. Driver, In the making	12
3.1.1. Modifying <code>main.c</code>	12
3.1.2. Modifying <code>device_file.c</code>	12
3.1.3. First run	15
3.2. C kernel reader, In the making	17
3.3. Conclutions	19
3.3.1. Commands that is used	19
3.3.2. Core Concepts	19
3.3.2.1. Character Device Registration and The Mall Analogy	19
3.3.2.2. Memory Isolation and Data Transfer Functions	20
4. Trying the GPIO	21
4.1. LED Controlling	21
4.1.1. GPIO Headers and Module Parameters	21
4.1.2. Device Write Handler	21
4.1.3. File Operations Mapping	22
4.1.4. GPIO Lifecycle Management	22
4.2. Nokia Screen SPI	23
4.2.1. Planning	23
4.2.2. SPI Research and Bit-Banging Mechanics	23
4.2.3. Backlit On and Off Control	24
4.2.4. Double Buffer Implementation	24
4.2.5. Smart Text Wrapping Implementation	25
4.2.6. Final Boss SPI screen	26
5. Conclusions	36

1. Before we start

This is the second logbook for RealTime Operating System. This assignment goals is to make us understand more about making a Linux Driver.

And this is my logbook of trying to make things work

2. Trying out the sample

2.1. The sample driver from Apriorit

from the assignment, we have to try out the sample linux driver from Apriorit. And this is what i get from the first simple driver explanation in the web (also in GitHub).

```
#include "device_file.h"
#include <linux/init.h>      /* module_init, module_exit */
#include <linux/module.h>    /* version info, MODULE_LICENSE, MODULE_AUTHOR */

MODULE_DESCRIPTION("Simple Linux driver");
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Apriorit, Inc");

static int simple_driver_init(void)
{
    int result = 0;
    pr_notice("Simple-driver: Initialization started\n");

    result = register_device();
    if (result)
    {
        pr_notice("Simple-driver: Failed to register character device\n");
    }
    return result;
}

static void simple_driver_exit(void)
{
    pr_notice("Simple-driver: Exiting\n");
    unregister_device();
}

module_init(simple_driver_init);
module_exit(simple_driver_exit);
```

Let me explain a bit of what is happening here :

- **module_init** → This is the function that tell the driver module loader (`insmod`, insert module) the function to run. In this case it would run the `simple_driver_init` function
- **module_exit** → This is the function that tell the driver module remover (`rmmod`, remove module) the function to run. In this case it would run the `simple_driver_exit` function
- **MODULES (anything with this in front)** → This is the module descriptor, so if you do `modinfo` you can see the description listed there.

- **pr_notice** → This is basically like the printf of your normal c file, but this print the notice in your dmesg (fun fact about pr_notice after this)

2.1.1. `pr_notice` fun fact :

```
printk(KERN_NOTICE "Simple-driver: Initialization started\n");  
pr_notice("Simple-driver: Initialization started\n");
```

this two are the same thing, so `pr_notice` is just `printk` with `KERN_NOTICE`. For me personally, i would just use `printk` because of muscle memory to type `printf`

But then after some searching, there is actually more to put in the `printk` function

- `KERN_EMERG` ⇒ System is unusable.
- `KERN_ERR` ⇒ Error conditions (usually used if `register_device()` fails).
- `KERN_WARNING` ⇒ Warning flags.
- `KERN_INFO` ⇒ Informational messages tracking healthy runtime behaviors.
- `KERN_NOTICE` ⇒ Normal but significant operational alerts.

The code is too long for me to put it here, so you can just open the code from https://github.com/apriorit/SimpleLinuxDriver/blob/master/device_file.c

Let me break it down for you :

```
int register_device(void) {  
    int result = 0;  
  
    pr_notice("Simple-driver: register_device() is called.\n");  
  
    unsigned baseminor = 0;  
    unsigned minor_count_required = 1;  
    result = alloc_chrdev_region(&g_devno, baseminor, minor_count_required,  
device_name);  
    if (result)  
    {  
        pr_err("Simple-driver: alloc_chrdev_region failed: %d\n", result);  
        goto err_out;  
    }  
  
    cdev_init(&g_cdev, &simple_driver_fops);  
    g_cdev.owner = THIS_MODULE;  
    result = cdev_add(&g_cdev, g_devno, minor_count_required);  
    if (result)  
    {  
        pr_err("Simple-driver: cdev_add failed: %d\n", result);  
        goto err_unregister_chrdev_region;  
    }  
  
    g_class = class_create(class_name);  
    if (IS_ERR(g_class))  
    {  
        result = PTR_ERR(g_class);  
    }  
}
```

```
    pr_err("Simple-driver: class_create failed: %d\n", result);
    goto err_cdev_del;
}

g_device = device_create(g_class, NULL, g_devno, NULL, device_name);
if (IS_ERR(g_device))
{
    result = PTR_ERR(g_device);
    pr_err("Simple-driver: device_create failed: %d\n", result);
    goto err_class_destroy;
}

pr_notice("Simple-driver: Registered character device with major number =
%i, minor number = %i\n", MAJOR(g_devno), MINOR(g_devno));
return 0;

err_class_destroy:
if (!IS_ERR_OR_NULL(g_class))
{
    class_destroy(g_class);
    g_class = NULL;
}

err_cdev_del:
cdev_del(&g_cdev);

err_unregister_chrdev_region:
if (g_devno)
{
    unsigned minor_count_allocated = 1;
    unregister_chrdev_region(g_devno, minor_count_allocated);
    g_devno = 0;
}

err_out:
return result;
}
```

This function is set for the module_init calling. In this function there is :

- 1. Allocate the identification (`alloc_chrdev_region`)
- 2. Initialize the internal kernel structure (`cdev_add`)
- 3. Create the system class functions (`class_create`)
- 4. Mount the user interface device file for I/O (`device_create`)

But then why is the code so long ? **Because of the fallback that we have**

Imagine if the code is crashing in step 4, you cannot just stop there, you have to clear all the steps before. Therefore you use a `goto` functions and make `err_`

functions to handle the clear up for you. So this initializer function really make sure that you didn't leak any memory or anything bad that we don't want.

```
void unregister_device(void)
{
    pr_notice("Simple-driver: unregister_device() is called\n");

    if (!IS_ERR_OR_NULL(g_device))
    {
        device_destroy(g_class, g_devno);
        g_device = NULL;
    }

    if (!IS_ERR_OR_NULL(g_class))
    {
        class_destroy(g_class);
        g_class = NULL;
    }

    cdev_del(&g_cdev);

    if (g_devno)
    {
        unsigned minor_count_allocated = 1;
        unregister_chrdev_region(g_devno, minor_count_allocated);
        g_devno = 0;
    }
    pr_info("Simple-driver: Unregistered\n");
}
```

This is the function that was called to remove the kernel module from the loaded kernel module (in other word `rmmod`). Here there is not much thing to explain. Just the same way like when you do the error handling in the `register_device` on top.

And the rest of the code

```
static const char g_s_Hello_World_string[] = "Hello world from kernel mode!\n";
static const ssize_t g_s_Hello_World_size = sizeof(g_s_Hello_World_string);

static ssize_t device_file_read(
    struct file *file_ptr, char __user *user_buffer, size_t count, loff_t
    *position)
{
    pr_notice("Simple-driver: Read from device file offset = %i, read bytes
    count = %u\n", (int)*position, (unsigned int)count);
```

```
if (*position ≥ g_s_Hello_World_size)
    return 0;

if (*position + count > g_s_Hello_World_size)
    count = g_s_Hello_World_size - *position;

if (copy_to_user(user_buffer, g_s_Hello_World_string + *position, count) != 0)
    return -EFAULT;

*position += count;
return count;
}
```

When analyzing the runtime behavior using the `dmesg` tool, I noticed the driver outputs two lines showing the offset and read bytes count. I broke down what these numbers actually mean during the file access loop:

- **Read Bytes Count:** This value represents the maximum size of the memory buffer that the application (in this case, the `cat` command) has allocated to hold incoming data. The system defaults this to `4096` bytes because that is the standard size of a single memory page in Linux. The utility requests this full amount even though our message is much smaller, simply because it does not know the file's size in advance.
- **Offset:** This serves as a positional tracking cursor or bookmark. It tells the driver exactly which byte index to start reading from inside the kernel string array. An offset of `0` means reading starts at the first character.
- **The Dual-Read Loop Mechanism:** The log records two distinct access attempts because of how the reading loop operates in the background:
 1. **First Pass (`offset = 0`):** The tool requests data from the beginning. The driver copies the 30-byte message over and shifts the tracking offset position directly to index 30.
 2. **Second Pass (`offset = 30`):** The tool loops back to verify if more text exists. It requests more data starting at position 30. The driver checks the boundaries, sees the offset has reached the end of the text, and returns a value of `0`. This zero return acts as the standard End-of-File signal, telling the reading tool to stop and exit safely.

But, how to tell the operating system which code to execute when we run the kernel? To explain how the operating system knows which code to execute when we run commands, we have to look at the `simple_driver_fops` structure.

```
static const struct file_operations simple_driver_fops =
{
```

C

```
.owner = THIS_MODULE,  
.read = device_file_read,  
};  
  
static const char device_name[] = "simple-driver";  
static const char class_name[] = "simple-driver-class";  
dev_t g_devno = 0;  
struct cdev g_cdev = {};  
static struct class *g_class = NULL;  
static struct device *g_device = NULL;
```

To make this mechanism easy to visualize, think of this structure as a **Standardized Services Menu** at a bank counter:

- A customer walking up to the bank window represents a User Space program (like the `cat` command).
- The customer cannot just walk into the secure vault or tell the bank tellers to run custom errands. They have to choose a standard operation directly from the laminated service menu sitting on the counter.

I broke down how this structure maps those menu choices to our actual backend driver instructions:

- **The VFS Bridge (Treating Devices like Files):** In Linux, there is a core design rule: “Everything is a file.” When a terminal command tries to talk to our device path at `/dev/simple-driver`, it passes through a system boundary called the Virtual File System (VFS). The VFS picks up the request and instantly checks our `file_operations` structure to see what specific rules and actions this custom device file is allowed to support.
- **Module Safety Allocation (`.owner = THIS_MODULE`):** This line acts as a building safety lock. It tells the operating system exactly which module owns this operational menu
- **Function Redirection (`.read = device_file_read`):** This is the literal line that glues the standard operating system read action to our custom C function. When the user selects the “Read” option from the system menu, the VFS looks at this pointer and redirects the execution straight down into our `device_file_read` function. Because we did not define any `.write` or `.open` pathways on this menu, the Linux kernel automatically handles those missing choices with built-in safety rejections. This completely blocks any write requests by default and turns our code into a secure, read-only device.

2.2. The Conclusions with Apriorit

Basically, when we compile it we will get like a `.ko` file and that is the one that you can do `insmod`. The github that we can pull from apriorit actually have this in the cmake

```
TARGET_MODULE:=simple-module
```

CMAKE

```
# If we running by kernel building system
ifneq ($(KERNELRELEASE),)
    $(TARGET_MODULE)-objs := main.o device_file.o
    obj-m := $(TARGET_MODULE).o

# If we are running without kernel build system
else
    BUILDSYSTEM_DIR:=/lib/modules/$(shell uname -r)/build
    PWD:=$(shell pwd)

all :
# run kernel build system to make module
    $(MAKE) -C $(BUILDSYSTEM_DIR) M=$(PWD) modules

clean:
# run kernel build system to cleanup in current directory
    $(MAKE) -C $(BUILDSYSTEM_DIR) M=$(PWD) clean

load:
    insmod ./${TARGET_MODULE}.ko

unload:
    rmmod ./${TARGET_MODULE}.ko

endif
```

See the load and unload part, its using insmod and rmmod.

And from that, after we are done compiling it, it will make a `.ko` file which we can use for loading it to the active kernel module it will load the main.c (the first code that i gave on top), and it will utilize the module init in the device_file.c

3. Making my own device driver.

3.1. Driver, In the making

In this part of the logbook, i will continue with the assignment 1b which is to change the name to my own name `bastian-driver`

3.1.1. Modifying `main.c`

I will start from the main, here i change everything that says simple to NW1728 (my alias is `nw1728@np1770`) this is the full main code

```
#include "device_file.h"
#include <linux/init.h> /* module_init, module_exit */
#include <linux/module.h> /* version info, MODULE_LICENSE, MODULE_AUTHOR */

MODULE_DESCRIPTION("BastianEthan Linux Driver");
MODULE_LICENSE("GPL");
MODULE_AUTHOR("NW1728@NP1770");

static int NW1728_driver_init(void) {
    int result = 0;
    pr_notice("NW1728-driver: Initialization started\n");

    result = register_device();
    if (result) {
        pr_notice("NW1728-driver: Failed to register character device\n");
    }
    return result;
}

static void NW1728_driver_exit(void) {
    pr_notice("NW1728-driver: Exiting\n");
    unregister_device();
}

module_init(NW1728_driver_init);
module_exit(NW1728_driver_exit);
```

3.1.2. Modifying `device_file.c`

Now i am going to do the other part, which is `device_file.c`.

```
#include "device_file.h"
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/errno.h> /* error codes */
#include <linux/fs.h> /* file stuff */
#include <linux/module.h> /* THIS_MODULE */
```

```
#include <linux/printk.h> /* pr_* */
#include <linux/uaccess.h> /* copy_to_user() */

static const char g_s_Hello_World_string[] = "Hello world from kernel mode!
\n";
static const ssize_t g_s_Hello_World_size = sizeof(g_s_Hello_World_string);

static ssize_t device_file_read(struct file *file_ptr, char __user
*user_buffer,
                                size_t count, loff_t *position) {
    pr_notice("NW1728-driver: Read from device file offset = %i, read bytes "
              "count = %u\n",
              (int)*position, (unsigned int)count);

    if (*position ≥ g_s_Hello_World_size)
        return 0;

    if (*position + count > g_s_Hello_World_size)
        count = g_s_Hello_World_size - *position;

    if (copy_to_user(user_buffer, g_s_Hello_World_string + *position, count) ≠
0)
        return -EFAULT;

    *position += count;
    return count;
}

static const struct file_operations nw1728_driver_fops = {
    .owner = THIS_MODULE,
    .read = device_file_read,
};

static const char device_name[] = "nw1728-driver";
static const char class_name[] = "nw1728-driver-class";
dev_t g_devno = 0;
struct cdev g_cdev = {};
static struct class *g_class = NULL;
static struct device *g_device = NULL;

int register_device(void) {
    int result = 0;

    pr_notice("NW1728-driver: register_device() is called.\n");

    unsigned baseminor = 0;
    unsigned minor_count_required = 1;
```

```
result = alloc_chrdev_region(&g_devno, baseminor, minor_count_required,
                             device_name);
if (result) {
    pr_err("NW1728-driver: alloc_chrdev_region failed: %d\n", result);
    goto err_out;
}

cdev_init(&g_cdev, &nw1728_driver_fops);
g_cdev.owner = THIS_MODULE;
result = cdev_add(&g_cdev, g_devno, minor_count_required);
if (result) {
    pr_err("NW1728-driver: cdev_add failed: %d\n", result);
    goto err_unregister_chrdev_region;
}

g_class = class_create(class_name);
if (IS_ERR(g_class)) {
    result = PTR_ERR(g_class);
    pr_err("NW1728-driver: class_create failed: %d\n", result);
    goto err_cdev_del;
}

g_device = device_create(g_class, NULL, g_devno, NULL, device_name);
if (IS_ERR(g_device)) {
    result = PTR_ERR(g_device);
    pr_err("NW1728-driver: device_create failed: %d\n", result);
    goto err_class_destroy;
}

pr_notice("NW1728-driver: Registered character device with major number = "
          "%i, minor number = %i\n",
          MAJOR(g_devno), MINOR(g_devno));
return 0;

err_class_destroy:
if (!IS_ERR_OR_NULL(g_class)) {
    class_destroy(g_class);
    g_class = NULL;
}

err_cdev_del:
cdev_del(&g_cdev);

err_unregister_chrdev_region:
if (g_devno) {
    unsigned minor_count_allocated = 1;
    unregister_chrdev_region(g_devno, minor_count_allocated);
}
```

```
    g_devno = 0;
}

err_out:
    return result;
}

void unregister_device(void) {
    pr_notice("NW1728-driver: unregister_device() is called\n");

    if (!IS_ERR_OR_NULL(g_device)) {
        device_destroy(g_class, g_devno);
        g_device = NULL;
    }

    if (!IS_ERR_OR_NULL(g_class)) {
        class_destroy(g_class);
        g_class = NULL;
    }

    cdev_del(&g_cdev);

    if (g_devno) {
        unsigned minor_count_allocated = 1;
        unregister_chrdev_region(g_devno, minor_count_allocated);
        g_devno = 0;
    }
    pr_info("NW1728-driver: Unregistered\n");
}
```

3.1.3. First run

Nice, now everything is under my name. Lets try to run it with `make all` commands in the RPI 2B.

```
BASH
make -C /lib/modules/6.12.75+rpt-rpi-v7/build M=/home/nw1728/study/kernel/
SimpleLinuxDriver modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v7'
  CC [M] /home/nw1728/study/kernel/SimpleLinuxDriver/main.o
  CC [M] /home/nw1728/study/kernel/SimpleLinuxDriver/device_file.o
  LD [M] /home/nw1728/study/kernel/SimpleLinuxDriver/simple-module.o
  MODPOST /home/nw1728/study/kernel/SimpleLinuxDriver/Module.symvers
  CC [M] /home/nw1728/study/kernel/SimpleLinuxDriver/simple-module.mod.o
  CC [M] /home/nw1728/study/kernel/SimpleLinuxDriver/.module-common.o
  LD [M] /home/nw1728/study/kernel/SimpleLinuxDriver/simple-module.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v7'
```

But wait, why the name is SimpleLinuxDriver ?? Oh yeah, **THE MAKE FILE**, Aight im going to change the make file. I think i have to change the target.

```
TARGET_MODULE:=nw1728-module

# If we running by kernel building system
ifneq ($(KERNELRELEASE),)
    $(TARGET_MODULE)-objs := main.o device_file.o
    obj-m := $(TARGET_MODULE).o

# If we are running without kernel build system
else
    BUILDSYSTEM_DIR:=/lib/modules/$(shell uname -r)/build
    PWD:=$(shell pwd)

all :
# run kernel build system to make module
    $(MAKE) -C $(BUILDSYSTEM_DIR) M=$(PWD) modules

clean:
# run kernel build system to cleanup in current directory
    $(MAKE) -C $(BUILDSYSTEM_DIR) M=$(PWD) clean

load:
    insmod ./$(TARGET_MODULE).ko

unload:
    rmmod ./$(TARGET_MODULE).ko

endif
```

Okay, Now lets try to compile this one.

```
RPI-2B - ཨོཾ་མ་ཎི་པདྨེ་ཧུམ་ OM MANI PADME HUM 俺嘛呢叭咪吽
→ SimpleLinuxDriver git:(master) × make clean
make -C /lib/modules/6.12.75+rpt-rpi-v7/build M=/home/nw1728/study/kernel/
SimpleLinuxDriver clean
make[1]: Entering directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v7'
CLEAN    /home/nw1728/study/kernel/SimpleLinuxDriver/Module.symvers
make[1]: Leaving directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v7'
RPI-2B - ཨོཾ་མ་ཎི་པདྨེ་ཧུམ་ OM MANI PADME HUM 俺嘛呢叭咪吽
→ SimpleLinuxDriver git:(master) × make all
make -C /lib/modules/6.12.75+rpt-rpi-v7/build M=/home/nw1728/study/kernel/
SimpleLinuxDriver modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v7'
CC [M]   /home/nw1728/study/kernel/SimpleLinuxDriver/main.o
CC [M]   /home/nw1728/study/kernel/SimpleLinuxDriver/device_file.o
```

```
LD [M] /home/nw1728/study/kernel/SimpleLinuxDriver/nw1728-module.o
MODPOST /home/nw1728/study/kernel/SimpleLinuxDriver/Module.symvers
CC [M] /home/nw1728/study/kernel/SimpleLinuxDriver/nw1728-module.mod.o
CC [M] /home/nw1728/study/kernel/SimpleLinuxDriver/.module-common.o
LD [M] /home/nw1728/study/kernel/SimpleLinuxDriver/nw1728-module.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.75+rpt-rpi-v7'
```

Okay, now lets load it

```
RPI-2B - ཨོཾ་མ་ཎི་པདྨེ་ཧུམ་ OM MANI PADME HUM ཨུམ་ཎི་པདྨེ་ཧུམ་
→ SimpleLinuxDriver git:(master) × sudo make load
insmod ./nw1728-module.ko
RPI-2B - ཨོཾ་མ་ཎི་པདྨེ་ཧུམ་ OM MANI PADME HUM ཨུམ་ཎི་པདྨེ་ཧུམ་
→ SimpleLinuxDriver git:(master) × dmesg | tail -n 3
[141564.857824] NW1728-driver: Initialization started
[141564.857859] NW1728-driver: register_device() is called.
[141564.866247] NW1728-driver: Registered character device with major number =
239, minor number = 0
```

Okay lets check if its working or not

```
RPI-2B - ཨོཾ་མ་ཎི་པདྨེ་ཧུམ་ OM MANI PADME HUM ཨུམ་ཎི་པདྨེ་ཧུམ་
→ SimpleLinuxDriver git:(master) × ls -la /dev/* | grep 'driver'
crw----- 1 root root 239, 0 Jun 18 15:09 /dev/nw1728-driver
lrwxrwxrwx 1 root root 16 Jun 18 15:09 239:0 -> ../nw1728-driver
RPI-2B - ཨོཾ་མ་ཎི་པདྨེ་ཧུམ་ OM MANI PADME HUM ཨུམ་ཎི་པདྨེ་ཧུམ་
→ SimpleLinuxDriver git:(master) × sudo cat > /dev/nw1728-driver
zsh: permission denied: /dev/nw1728-driver
RPI-2B - ཨོཾ་མ་ཎི་པདྨེ་ཧུམ་ OM MANI PADME HUM ཨུམ་ཎི་པདྨེ་ཧུམ་
→ SimpleLinuxDriver git:(master) × sudo chmod 666 /dev/nw1728-driver
RPI-2B - ཨོཾ་མ་ཎི་པདྨེ་ཧུམ་ OM MANI PADME HUM ཨུམ་ཎི་པདྨེ་ཧུམ་
→ SimpleLinuxDriver git:(master) × sudo cat < /dev/nw1728-driver
Hello world from kernel mode!
```

Great Success, -borat

3.2. C kernel reader, In the making

The wise one once typed

“Make also a small test programme in C where you open the device as a file, read from it character for character and displaying them using printf.”

So i am going to do exactly what the wise one told me to do

C

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

#define DEVICE_PATH "/dev/nw1728-driver"

int main(void)
{
    // Open the device file in read-only mode
    int fd = open(DEVICE_PATH, O_RDONLY);
    if (fd < 0)
    {
        perror("Error: Cannot open device file. Did you load the module and
are you root?");
        return EXIT_FAILURE;
    }

    printf("Successfully opened %s\n", DEVICE_PATH);
    printf("--- Reading contents character by character ---\n");

    char ch;
    ssize_t bytes_read;

    // Read exactly 1 byte at a time until end-of-file (0) or error (-1)
    while ((bytes_read = read(fd, &ch, 1)) > 0)
    {
        // Flush stdout immediately to see character-by-character progression
        printf("%c", ch);
        fflush(stdout);
    }

    if (bytes_read < 0)
    {
        perror("\nError occurred while reading from device");
        close(fd);
        return EXIT_FAILURE;
    }

    printf("\n--- End of File reached ---\n");

    // Clean up resource descriptor
    close(fd);
    return EXIT_SUCCESS;
}
```

And now lets try to compile and run it

```
RPI-2B - ཨོཾ་མ་ཎི་པདྨེ་ཧུམ་ OM MANI PADME HUM 唵嘛呢叭咪吽
→ SimpleLinuxDriver git:(master) × gcc test_client.c -o test_client
RPI-2B - ཨོཾ་མ་ཎི་པདྨེ་ཧུམ་ OM MANI PADME HUM 唵嘛呢叭咪吽
→ SimpleLinuxDriver git:(master) × ./test_client
Successfully opened /dev/nw1728-driver
--- Reading contents character by character ---
Hello world from kernel mode!

--- End of File reached ---
```

Nice, it works again.

Now lets play with GPIO Pins (or what Raspberry call HAT+)

3.3. Conclutions

3.3.1. Commands that is used

- Compilation : `make`
 - **Why it is used:** Compiles the source files (`main.c` and `device_file.c`) and generates the kernel module object file (`nw1728-module.ko`).
- Installation: `sudo insmod nw1728-module.ko`
 - **Why it is used:** “Insert Module” dynamically loads our `.ko` module into the running kernel.
- Execution/Testing: Writing and Reading the Device File
 - **Why it is used:** We use standard file utilities like `echo` and `tee` to write control signals to the driver, or `cat` to read from it.
 - **Commands:**
 - To Turn LED ON: `echo "1" | sudo tee /dev/nw1728-driver`
 - To Turn LED OFF: `echo "0" | sudo tee /dev/nw1728-driver`
 - To Read Hello Message: `cat /dev/nw1728-driver`
- Uninstallation: `sudo rmmod nw1728-module`
 - **Why it is used:** “Remove Module” unloads the driver from the kernel.

3.3.2. Core Concepts

3.3.2.1. Character Device Registration and The Mall Analogy

In older Linux drivers, developers used `register_chrdev()`, which locked down an entire major number and all 256 minor numbers at once.

Modern drivers fix this waste by splitting the setup into a 3-step dynamic process, which can be compared to opening a new business kiosk inside a **Massive Shopping Mall**.

- **The Old Way (`register_chrdev`) Renting a Whole Floor:** This legacy function is like renting an entire mall floor just to set up one small information desk. It completely blocks all 256 shop slots on that floor, quickly running out of available major numbers.

- **Step 1: `alloc_chrdev_region` Reserving Your Room Number:** This modern function dynamically requests a single, unique room address (Major and Minor number combination) from the kernel. In our analogy, mall management checks their database and assigns you to **Floor 240, Room 0**, leaving the other rooms open.
- **Step 2: `cdev_init` & `cdev_add` Building the Service Counter:** This step initializes your character device structure and binds it to your file operations code map. This is like physically assembling your desk inside your assigned room and training your staff, though the public hallway doors are still locked.
- **Step 3: `class_create` & `device_create` Adding to the Mall Directory:** These functions create a class directory in sysfs and generate the active device file node at `/dev/nw1728-driver`. This acts like listing your business on the main lobby directory board and carving a physical service window through the lobby wall for customers.

3.3.2.2. Memory Isolation and Data Transfer Functions

Linux splits system memory into two isolated zones to keep the operating system secure. To understand this separation, we can imagine Kernel-Space as a **High-Security Government Vault** and User-Space as the **Public Lobby outside the building**.

- **User-Space Memory:** This is where normal programs run in their own isolated bubbles. If a user application crashes, it only kills that single program without hurting the rest of the system.
- **Kernel-Space Memory:** This zone has absolute control over the hardware, CPU, and physical RAM blocks. If code here encounters a bad memory pointer, the entire operating system crashes instantly into a Kernel Panic.
- **The Privilege Barrier:** Because of this strict barrier, kernel code cannot directly read or write to pointers handed over by user apps. The system requires specialized, secure transfer windows to pass data safely across the line.
- **`copy_to_user()` — Exporting Data to the Public Lobby:** This function securely copies data out of privileged kernel memory and hands it to a user-space application buffer. In our driver's read function, it acts like a secure tray passing the "Hello world..." string out to the terminal command.
- **`copy_from_user()` — Importing Data into the Vault:** This function pulls data from an untrusted user-space application and places it safely into a kernel memory buffer. For our upcoming write functions, it checks and imports the user's input command byte to toggle the hardware state safely.

4. Trying the GPIO

Okay now lets do the extra part, lets keep thing simple first, only controlling 1 led at a time.

4.1. LED Controlling

All modification is done only in device_file.c

4.1.1. GPIO Headers and Module Parameters

We import the kernel's GPIO header and define a module parameter. One cool thing, i am using a parameter that allows me to specify which GPIO pin to use dynamically when inserting the module (e.g., `insmod nw1728-module.ko gpio_pin=X`).

```
#include <linux/gpio.h> /* gpio_request, gpio_set_value, etc. */

static unsigned int gpio_pin = 533; /* Default GPIO 21 (base 512 + 21) */
module_param(gpio_pin, uint, 0644);
MODULE_PARM_DESC(gpio_pin, "GPIO pin number to control (default 21)");
```

4.1.2. Device Write Handler

I implemented `device_file_write` to process data written to the device file `/dev/nw1728-driver`. It reads the first character from the user-space buffer, parses it, and drives the GPIO state.

```
static ssize_t device_file_write(struct file *file_ptr, const char __user
*user_buffer,
                                size_t count, loff_t *position) {
    char value_char;

    if (count == 0)
        return 0;

    // Copy data safely from user-space to kernel-space
    if (copy_from_user(&value_char, user_buffer, 1) != 0) {
        return -EFAULT;
    }

    // Toggle GPIO pin voltage levels
    if (value_char == '1') {
        gpio_set_value(gpio_pin, 1); // High (3.3V)
        pr_notice("NW1728-driver: LED turned ON (GPIO %u)\n", gpio_pin);
    } else if (value_char == '0') {
        gpio_set_value(gpio_pin, 0); // Low (GND)
        pr_notice("NW1728-driver: LED turned OFF (GPIO %u)\n", gpio_pin);
    } else {
        pr_notice("NW1728-driver: Invalid write value '%c' (only '1' or '0'
accepted)\n", value_char);
    }
}
```

```
    }  
  
    return count;  
}
```

4.1.3. File Operations Mapping

I updated the `file_operations` structure so that the driver knows what to do if we put something to the Driver File (the `/dev/nw1728-driver`)

```
static const struct file_operations nw1728_driver_fops = {  
    .owner = THIS_MODULE,  
    .read = device_file_read,  
    .write = device_file_write, // New write mapping  
};
```

4.1.4. GPIO Lifecycle Management

I allocate and initialize the GPIO pin inside the `register_device` function and free it inside the `unregister_device` function (as well as inside the error labels of `register_device`).

Initialization (`register_device`):

```
// Validate if the requested pin is valid  
if (!gpio_is_valid(gpio_pin)) {  
    pr_err("NW1728-driver: Invalid GPIO pin %u requested\n", gpio_pin);  
    result = -EINVAL;  
    goto err_out;  
}  
  
// Request GPIO ownership from the kernel  
result = gpio_request(gpio_pin, "nw1728-led");  
if (result) {  
    pr_err("NW1728-driver: gpio_request failed for pin %u: %d\n", gpio_pin,  
result);  
    goto err_out;  
}  
  
// Set the pin direction to output (and initialize it to OFF / Low state)  
result = gpio_direction_output(gpio_pin, 0);  
if (result) {  
    pr_err("NW1728-driver: gpio_direction_output failed for pin %u: %d\n",  
gpio_pin, result);  
    goto err_free_gpio;  
}
```

Cleanup & Error Paths:

```
// Inside unregister_device and error handler labels:  
gpio_free(gpio_pin);
```

4.2. Nokia Screen SPI

With the LED being done now, lets cook some stuff

From The wise man (the one that tell me to do a C file to interact with the kernel driver), he lended me a very unique stuff. Its a Nokia 5110 Screen, but with an SPI breakout board, made by Adafruit.

<https://github.com/adafruit/Adafruit-PCD8544-Nokia-5110-LCD-library.git>

Its a neat little screen. Also very retro looking. But now we have work to do, which is to make the screen work

4.2.1. Planning

Let's do the planning. Here is some of my plan :

- I want the screen to be connected with SPI to the GPIO (Pi Hat+)
- I want the SPI screen to be initialized when we load the module (then initializer in the register device func)
- I want the Buffer to change as well (whats on the screen = whats on the buffer)
- I want the backlit to turn on when the RPI turn on, and turn off when the RPI turned off (power indicator logic)
- I want when i do `echo` something to the driver, the screen update immediately
- I want the text wrapping to not cut any words (so it doesn't split "and" to be "a" on top and "nd" on the bottom)

I think this enough for Planning. Now lets cook.

4.2.2. SPI Research and Bit-Banging Mechanics

To get our Nokia 5110 screen working, we are using a technique called **SPI Bit-Banging**. Instead of relying on the Raspberry Pi's built-in hardware SPI controller (A very nice way to torture yourself for the masochist out there), our code manually flips regular GPIO pins up and down very quickly to mimic the SPI communication protocol.

- **The Telephone Analogy for Bit-Banging:** Imagine trying to send a text message to a friend by flashing a flashlight out your window. Hardware SPI is like using a laser machine that automatically pulses the light at high speed, while bit-banging is like manually clicking the flashlight switch with your thumb for every single dot and dash. It takes more CPU effort, but it allows us to turn any generic GPIO pins into an active data pipeline.
- **SPI Communication Lines:** Our code maps out five distinct connections to talk to the screen module:
 - ▶ `pin_clk` (Clock): The heartbeat line that tells the screen exactly when to look for a new incoming data bit.
 - ▶ `pin_din` (Data In): The delivery line that carries the actual data bits (0 or 1) into the display.

- ▶ `pin_dc` (Data/Command Control): A toggle line that tells the screen if the incoming byte is a setting adjustment instruction or a literal text pixel.
- ▶ `pin_cs` (Chip Select): An activation line that turns the screen's listener interface on or off.
- ▶ `pin_rst` (Hardware Reset): A startup line that clears the screen's internal memory when our module initializes.
- **The Bit-Bang Loop Mechanism (`spi_write_byte`):** The driver sends a single 8-bit block of data by using a simple `for` loop that runs exactly 8 times. For each bit, it pulls the clock low, checks whether the current bit is a 0 or 1 to set the data line, and then pulls the clock high to force the screen to read it.
- **The Stacked Unwinding Expansion (`register_device`):** Our initialization function has grown much longer because we added five hardware pins to the setup process. Since every pin must be requested using `gpio_request()`, our cascading fallback path at the bottom has extra `err_free` labels to safely drop each pin in reverse order if a downstream allocation fails.

4.2.3. Backlit On and Off Control

Instead of writing a bunch of complex C code inside our driver just to turn on the screen's backlight, we can let the Raspberry Pi hardware handle it automatically. We can do this by adding one simple configuration line to the Pi's startup files.

- **The way i do it:** We just add `gpio=14=op,dh` to the `/boot/firmware/config.txt` file. This tells the Pi the very second it boots up to turn GPIO pin 14 into an output pin (`op`) and flip its power to high (`dh`), which turns the light on.

4.2.4. Double Buffer Implementation

To make our screen work properly, we implemented a double-buffer system. This means when a user reads from our driver using `cat`, the driver returns the exact text currently shown on the screen, instead of a static welcome message.

- **The Dual Buffer Design:** Our code handles this using two distinct storage arrays in memory:

```
static uint8_t lcd_buf[LCD_WIDTH * LCD_PAGES];  
static char lcd_text_buf[513];  
static size_t lcd_text_len;
```

`lcd_text_buf` holds the raw ASCII text characters (up to 512 bytes) that the user writes to the driver. `lcd_buf` holds the raw, calculated pixel blocks (84x48 bits) generated by our font mapper. When you echo text to the driver, `device_file_write` safely catches it, saves it inside `lcd_text_buf`, and triggers the screen to redraw the pixels:

```
if (copy_from_user(buf, user_buffer, count)) {  
    kfree(buf);  
}
```

```
        return -EFAULT;
    }
    buf[count] = '\0';

    mutex_lock(&lcd_lock);
    memcpy(lcd_text_buf, buf, count);
    lcd_text_buf[count] = '\0';
    lcd_text_len = count;
    lcd_draw_text(buf, count);
    mutex_unlock(&lcd_lock);
```

When you run `cat`, `device_file_read` hooks straight into that exact same `lcd_text_buf` memory space, handing back the last written message so your terminal and screen stay perfectly synchronized:

```
mutex_lock(&lcd_lock);
if (*position ≥ (loff_t)lcd_text_len) {
    mutex_unlock(&lcd_lock);
    return 0;
}
if (*position + count > lcd_text_len)
    count = lcd_text_len - *position;
if (copy_to_user(user_buffer, lcd_text_buf + *position, count) ≠ 0) {
    mutex_unlock(&lcd_lock);
    return -EFAULT;
}
*position += count;
ret = count;
mutex_unlock(&lcd_lock);
```

4.2.5. Smart Text Wrapping Implementation

To stop words from getting chopped in half at the edges of our small 84-pixel screen, we added a smart text-wrapping rule. This keeps whole words together so they look clean and readable.

- **The Comic Strip Analogy:** Imagine writing a comic strip by hand inside tiny boxes. If you are typing a long word like “Raspberry” and realize you only have space for “Rasp” at the end of the line, you wouldn’t split it. Instead, you would smash the eraser, skip that tiny empty space, and write the whole word “Raspberry” neatly on the next line. That is exactly what our driver does.
- **The Peeking Ahead Code:** Before drawing a letter, the driver checks if it is the start of a new word. If it is, it quickly peeks ahead to count how long the word is and shifts down to a new line if it won’t fit:

```
/* first char of a word: wrap ahead if word fits on a fresh line */
if (word_start && col > 0) {
    size_t j = i, wlen = 0;
    while (j < len && text[j] != ' ' && text[j] != '\n')
        j++, wlen++;
    if (col + (int)wlen > LCD_COLS && (int)wlen <= LCD_COLS) {
        col = 0;
        row++;
        if (row >= LCD_PAGES)
            break;
        continue; /* re-process same char at new line */
    }
}
word_start = 0;
```

- **The Safety Fallback:** If a user types a single word that is way too giant for the whole screen (more than 14 letters long), peeking ahead won't save us. The driver uses a backup safety rule to just cut the word anyway so the system doesn't freeze or crash:

```
/* hard wrap fallback for words longer than LCD_COLS */
if (col >= LCD_COLS) {
    col = 0;
    row++;
    if (row >= LCD_PAGES)
        break;
}

lcd_draw_char(col, row, c);
col++;
i++;
```

If a user types a word that is completely longer than the width of the entire screen (more than 14 characters), a lookahead wrap is impossible. In this specific case, the driver falls back to a clean hard-wrap to prevent memory corruption:

4.2.6. Final Boss SPI screen

After everything that i already done. Here is the code for the SPI Screen

```
#include "device_file.h"
#include <linux/cdev.h>
#include <linux/delay.h>
#include <linux/device.h>
#include <linux/errno.h>
#include <linux/fs.h>
```

```
#include <linux/gpio.h>
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/printk.h>
#include <linux/slab.h>
#include <linux/string.h>
#include <linux/uaccess.h>

/* GPIO pin numbers: kernel absolute = gpiochip base (512 on Pi) + BCM number
*/
static unsigned int pin_clk = 523; /* BCM 11 - SPI clock */
static unsigned int pin_din = 522; /* BCM 10 - SPI data */
static unsigned int pin_dc = 537; /* BCM 25 - data/cmd */
static unsigned int pin_cs = 520; /* BCM 8 - chip select */
static unsigned int pin_rst = 536; /* BCM 24 - reset */

module_param(pin_clk, uint, 0644);
module_param(pin_din, uint, 0644);
module_param(pin_dc, uint, 0644);
module_param(pin_cs, uint, 0644);
module_param(pin_rst, uint, 0644);

MODULE_PARM_DESC(pin_clk, "GPIO absolute number for CLK (default 523 = BCM 11)");
MODULE_PARM_DESC(pin_din, "GPIO absolute number for DIN (default 522 = BCM 10)");
MODULE_PARM_DESC(pin_dc, "GPIO absolute number for DC (default 537 = BCM 25)");
MODULE_PARM_DESC(pin_cs, "GPIO absolute number for CS (default 520 = BCM 8)");
MODULE_PARM_DESC(pin_rst, "GPIO absolute number for RST (default 536 = BCM 24)");

#define LCD_WIDTH 84
#define LCD_HEIGHT 48
#define LCD_PAGES 6 /* LCD_HEIGHT / 8 */
#define LCD_COLS 14 /* LCD_WIDTH / 6 (5px char + 1px gap) */

static uint8_t lcd_buf[LCD_WIDTH * LCD_PAGES];
static DEFINE_MUTEX(lcd_lock);

/* ----- */
/* SPI bit-bang (Mode 0: CLK idles low, data latched on rising edge) */
/* ----- */

static void spi_write_byte(uint8_t byte)
{

```

```
int i;
for (i = 7; i ≥ 0; i--) {
    gpio_set_value(pin_clk, 0);
    gpio_set_value(pin_din, (byte >> i) & 1);
    gpio_set_value(pin_clk, 1);
}
gpio_set_value(pin_clk, 0);
}

static void lcd_cmd(uint8_t cmd)
{
    gpio_set_value(pin_dc, 0);
    gpio_set_value(pin_cs, 0);
    spi_write_byte(cmd);
    gpio_set_value(pin_cs, 1);
}

/* ----- */
/* PCD8544 (Nokia 5110) display control */
/* ----- */

static void lcd_init(void)
{
    gpio_set_value(pin_rst, 0);
    udelay(1000); /* 1 ms reset pulse (PCD8544 needs ≥100 ns) */
    gpio_set_value(pin_rst, 1);
    udelay(5000); /* settle before first command */

    lcd_cmd(0x21); /* function set: extended instruction set */
    lcd_cmd(0x14); /* set bias: 1:48 ratio (bias mode 4) */
    lcd_cmd(0xBC); /* set Vop (contrast): 0x80 | 60 */
    lcd_cmd(0x20); /* function set: basic instruction set */
    lcd_cmd(0x0C); /* display control: normal mode */
}

/* Push the entire framebuffer to the display */
static void lcd_flush(void)
{
    int page, col;
    for (page = 0; page < LCD_PAGES; page++) {
        lcd_cmd(0x40 | page); /* SETYADDR: select page 0-5 */
        lcd_cmd(0x80); /* SETXADDR: column 0 */
        gpio_set_value(pin_dc, 1);
        gpio_set_value(pin_cs, 0);
        for (col = 0; col < LCD_WIDTH; col++)
            spi_write_byte(lcd_buf[page * LCD_WIDTH + col]);
        gpio_set_value(pin_cs, 1);
    }
}
```

```
    }
    lcd_cmd(0x40);          /* reset Y address to 0 */
}

static void lcd_clear(void)
{
    memset(lcd_buf, 0, sizeof(lcd_buf));
}

/* ----- */
/* 5x8 bitmap font (ASCII 32-126) */
/* Each entry: 5 column bytes, bit 0 = top row */
/* ----- */

static const uint8_t font5x8[][5] = {
    { 0x00, 0x00, 0x00, 0x00, 0x00 }, /* 32 */
    { 0x00, 0x00, 0x5F, 0x00, 0x00 }, /* 33 ! */
    { 0x00, 0x07, 0x00, 0x07, 0x00 }, /* 34 " */
    { 0x14, 0x7F, 0x14, 0x7F, 0x14 }, /* 35 # */
    { 0x24, 0x2A, 0x7F, 0x2A, 0x12 }, /* 36 $ */
    { 0x23, 0x13, 0x08, 0x64, 0x62 }, /* 37 % */
    { 0x36, 0x49, 0x55, 0x22, 0x50 }, /* 38 & */
    { 0x00, 0x05, 0x03, 0x00, 0x00 }, /* 39 ' */
    { 0x00, 0x1C, 0x22, 0x41, 0x00 }, /* 40 ( */
    { 0x00, 0x41, 0x22, 0x1C, 0x00 }, /* 41 ) */
    { 0x0A, 0x04, 0x1F, 0x04, 0x0A }, /* 42 * */
    { 0x08, 0x08, 0x3E, 0x08, 0x08 }, /* 43 + */
    { 0x00, 0x50, 0x30, 0x00, 0x00 }, /* 44 , */
    { 0x08, 0x08, 0x08, 0x08, 0x08 }, /* 45 - */
    { 0x00, 0x60, 0x60, 0x00, 0x00 }, /* 46 . */
    { 0x20, 0x10, 0x08, 0x04, 0x02 }, /* 47 / */
    { 0x3E, 0x51, 0x49, 0x45, 0x3E }, /* 48 0 */
    { 0x00, 0x42, 0x7F, 0x40, 0x00 }, /* 49 1 */
    { 0x42, 0x61, 0x51, 0x49, 0x46 }, /* 50 2 */
    { 0x21, 0x41, 0x45, 0x4B, 0x31 }, /* 51 3 */
    { 0x18, 0x14, 0x12, 0x7F, 0x10 }, /* 52 4 */
    { 0x27, 0x45, 0x45, 0x45, 0x39 }, /* 53 5 */
    { 0x3C, 0x4A, 0x49, 0x49, 0x30 }, /* 54 6 */
    { 0x01, 0x71, 0x09, 0x05, 0x03 }, /* 55 7 */
    { 0x36, 0x49, 0x49, 0x49, 0x36 }, /* 56 8 */
    { 0x06, 0x49, 0x49, 0x29, 0x1E }, /* 57 9 */
    { 0x00, 0x36, 0x36, 0x00, 0x00 }, /* 58 : */
    { 0x00, 0x56, 0x36, 0x00, 0x00 }, /* 59 ; */
    { 0x00, 0x08, 0x14, 0x22, 0x41 }, /* 60 < */
    { 0x14, 0x14, 0x14, 0x14, 0x14 }, /* 61 = */
    { 0x41, 0x22, 0x14, 0x08, 0x00 }, /* 62 > */
    { 0x02, 0x01, 0x51, 0x09, 0x06 }, /* 63 ? */
}
```

```
{ 0x32, 0x49, 0x79, 0x41, 0x3E }, /* 64 @ */
{ 0x7E, 0x11, 0x11, 0x11, 0x7E }, /* 65 A */
{ 0x7F, 0x49, 0x49, 0x49, 0x36 }, /* 66 B */
{ 0x3E, 0x41, 0x41, 0x41, 0x22 }, /* 67 C */
{ 0x7F, 0x41, 0x41, 0x22, 0x1C }, /* 68 D */
{ 0x7F, 0x49, 0x49, 0x49, 0x41 }, /* 69 E */
{ 0x7F, 0x09, 0x09, 0x01, 0x01 }, /* 70 F */
{ 0x3E, 0x41, 0x41, 0x51, 0x32 }, /* 71 G */
{ 0x7F, 0x08, 0x08, 0x08, 0x7F }, /* 72 H */
{ 0x00, 0x41, 0x7F, 0x41, 0x00 }, /* 73 I */
{ 0x20, 0x40, 0x41, 0x3F, 0x01 }, /* 74 J */
{ 0x7F, 0x08, 0x14, 0x22, 0x41 }, /* 75 K */
{ 0x7F, 0x40, 0x40, 0x40, 0x40 }, /* 76 L */
{ 0x7F, 0x02, 0x04, 0x02, 0x7F }, /* 77 M */
{ 0x7F, 0x04, 0x08, 0x10, 0x7F }, /* 78 N */
{ 0x3E, 0x41, 0x41, 0x41, 0x3E }, /* 79 O */
{ 0x7F, 0x09, 0x09, 0x09, 0x06 }, /* 80 P */
{ 0x3E, 0x41, 0x51, 0x21, 0x5E }, /* 81 Q */
{ 0x7F, 0x09, 0x19, 0x29, 0x46 }, /* 82 R */
{ 0x46, 0x49, 0x49, 0x49, 0x31 }, /* 83 S */
{ 0x01, 0x01, 0x7F, 0x01, 0x01 }, /* 84 T */
{ 0x3F, 0x40, 0x40, 0x40, 0x3F }, /* 85 U */
{ 0x1F, 0x20, 0x40, 0x20, 0x1F }, /* 86 V */
{ 0x3F, 0x40, 0x38, 0x40, 0x3F }, /* 87 W */
{ 0x63, 0x14, 0x08, 0x14, 0x63 }, /* 88 X */
{ 0x03, 0x04, 0x78, 0x04, 0x03 }, /* 89 Y */
{ 0x61, 0x51, 0x49, 0x45, 0x43 }, /* 90 Z */
{ 0x00, 0x00, 0x7F, 0x41, 0x41 }, /* 91 [ */
{ 0x02, 0x04, 0x08, 0x10, 0x20 }, /* 92 \ */
{ 0x41, 0x41, 0x7F, 0x00, 0x00 }, /* 93 ] */
{ 0x04, 0x02, 0x01, 0x02, 0x04 }, /* 94 ^ */
{ 0x40, 0x40, 0x40, 0x40, 0x40 }, /* 95 _ */
{ 0x00, 0x01, 0x02, 0x04, 0x00 }, /* 96 ` */
{ 0x20, 0x54, 0x54, 0x54, 0x78 }, /* 97 a */
{ 0x7F, 0x48, 0x44, 0x44, 0x38 }, /* 98 b */
{ 0x38, 0x44, 0x44, 0x44, 0x20 }, /* 99 c */
{ 0x38, 0x44, 0x44, 0x48, 0x7F }, /* 100 d */
{ 0x38, 0x54, 0x54, 0x54, 0x18 }, /* 101 e */
{ 0x08, 0x7E, 0x09, 0x01, 0x02 }, /* 102 f */
{ 0x08, 0x14, 0x54, 0x54, 0x3C }, /* 103 g */
{ 0x7F, 0x08, 0x04, 0x04, 0x78 }, /* 104 h */
{ 0x00, 0x44, 0x7D, 0x40, 0x00 }, /* 105 i */
{ 0x20, 0x40, 0x44, 0x3D, 0x00 }, /* 106 j */
{ 0x00, 0x7F, 0x10, 0x28, 0x44 }, /* 107 k */
{ 0x00, 0x41, 0x7F, 0x40, 0x00 }, /* 108 l */
{ 0x7C, 0x04, 0x18, 0x04, 0x78 }, /* 109 m */
{ 0x7C, 0x08, 0x04, 0x04, 0x78 }, /* 110 n */
```

```
{ 0x38, 0x44, 0x44, 0x44, 0x38 }, /* 111 o */
{ 0x7C, 0x14, 0x14, 0x14, 0x08 }, /* 112 p */
{ 0x08, 0x14, 0x14, 0x18, 0x7C }, /* 113 q */
{ 0x7C, 0x08, 0x04, 0x04, 0x08 }, /* 114 r */
{ 0x48, 0x54, 0x54, 0x54, 0x20 }, /* 115 s */
{ 0x04, 0x3F, 0x44, 0x40, 0x20 }, /* 116 t */
{ 0x3C, 0x40, 0x40, 0x20, 0x7C }, /* 117 u */
{ 0x1C, 0x20, 0x40, 0x20, 0x1C }, /* 118 v */
{ 0x3C, 0x40, 0x30, 0x40, 0x3C }, /* 119 w */
{ 0x44, 0x28, 0x10, 0x28, 0x44 }, /* 120 x */
{ 0x0C, 0x50, 0x50, 0x50, 0x3C }, /* 121 y */
{ 0x44, 0x64, 0x54, 0x4C, 0x44 }, /* 122 z */
{ 0x00, 0x08, 0x36, 0x41, 0x00 }, /* 123 { */
{ 0x00, 0x00, 0x7F, 0x00, 0x00 }, /* 124 | */
{ 0x00, 0x41, 0x36, 0x08, 0x00 }, /* 125 } */
{ 0x08, 0x08, 0x2A, 0x1C, 0x08 }, /* 126 ~ */
};

/* Draw one ASCII character into lcd_buf at grid position (col, row) */
static void lcd_draw_char(int col, int row, char c)
{
    int i;
    int x = col * 6; /* 5px char + 1px gap */

    if (c < 32 || c > 126)
        c = '?';
    if (x + 5 > LCD_WIDTH || row ≥ LCD_PAGES)
        return;

    for (i = 0; i < 5; i++)
        lcd_buf[row * LCD_WIDTH + x + i] = font5x8[(uint8_t)c - 32][i];
    lcd_buf[row * LCD_WIDTH + x + 5] = 0x00; /* column gap */
}

/* Render a text string into lcd_buf, then push to display */
static void lcd_draw_text(const char *text, size_t len)
{
    int col = 0, row = 0;
    size_t i;

    lcd_clear();
    for (i = 0; i < len && row < LCD_PAGES; i++) {
        if (text[i] == '\n') {
            col = 0;
            row++;
            continue;
        }
    }
}
```

```
    if (col ≥ LCD_COLS) {
        col = 0;
        row++;
        if (row ≥ LCD_PAGES)
            break;
    }
    lcd_draw_char(col, row, text[i]);
    col++;
}
lcd_flush();
}

/* ----- */
/* Character device file operations */
/* ----- */

static ssize_t device_file_read(struct file *file_ptr, char __user
*user_buffer,
    size_t count, loff_t *position)
{
    static const char status[] = "Nokia 5110 PCD8544 driver ready\n";
    static const ssize_t status_len = sizeof(status) - 1;

    if (*position ≥ status_len)
        return 0;
    if (*position + count > status_len)
        count = status_len - *position;
    if (copy_to_user(user_buffer, status + *position, count) ≠ 0)
        return -EFAULT;
    *position += count;
    return count;
}

static ssize_t device_file_write(struct file *file_ptr,
    const char __user *user_buffer,
    size_t count, loff_t *position)
{
    char *buf;

    if (count == 0)
        return 0;
    if (count > 512)
        count = 512;

    buf = kmalloc(count + 1, GFP_KERNEL);
    if (!buf)
        return -ENOMEM;
}
```

```
if (copy_from_user(buf, user_buffer, count)) {
    kfree(buf);
    return -EFAULT;
}
buf[count] = '\\0';

mutex_lock(&lcd_lock);
lcd_draw_text(buf, count);
mutex_unlock(&lcd_lock);

pr_notice("NW1728-driver: LCD display updated: %s", buf);
kfree(buf);
return count;
}

static const struct file_operations nw1728_driver_fops = {
    .owner = THIS_MODULE,
    .read = device_file_read,
    .write = device_file_write,
};

/* ----- */
/* Device registration / teardown */
/* ----- */

static const char device_name[] = "nw1728-driver";
static const char class_name[] = "nw1728-driver-class";
dev_t g_devno = 0;
struct cdev g_cdev = {};
static struct class *g_class = NULL;
static struct device *g_device = NULL;

int register_device(void)
{
    int result = 0;
    unsigned baseminor = 0;
    unsigned minor_count = 1;

    pr_notice("NW1728-driver: register_device() called\\n");

    result = gpio_request(pin_clk, "nw1728-clk");
    if (result) { pr_err("NW1728-driver: gpio_request clk failed: %d\\n",
result); goto err_out; }
    gpio_direction_output(pin_clk, 0);

    result = gpio_request(pin_din, "nw1728-din");
```

```
    if (result) { pr_err("NW1728-driver: gpio_request din failed: %d\n",
result); goto err_free_clk; }
    gpio_direction_output(pin_din, 0);

    result = gpio_request(pin_dc, "nw1728-dc");
    if (result) { pr_err("NW1728-driver: gpio_request dc failed: %d\n", result);
goto err_free_din; }
    gpio_direction_output(pin_dc, 0);

    result = gpio_request(pin_cs, "nw1728-cs");
    if (result) { pr_err("NW1728-driver: gpio_request cs failed: %d\n", result);
goto err_free_dc; }
    gpio_direction_output(pin_cs, 1);

    result = gpio_request(pin_rst, "nw1728-rst");
    if (result) { pr_err("NW1728-driver: gpio_request rst failed: %d\n",
result); goto err_free_cs; }
    gpio_direction_output(pin_rst, 1);

    lcd_init();
    lcd_clear();
    lcd_flush();

    result = alloc_chrdev_region(&g_devno, baseminor, minor_count, device_name);
    if (result) {
        pr_err("NW1728-driver: alloc_chrdev_region failed: %d\n", result);
        goto err_free_gpio;
    }

    cdev_init(&g_cdev, &nw1728_driver_fops);
    g_cdev.owner = THIS_MODULE;
    result = cdev_add(&g_cdev, g_devno, minor_count);
    if (result) {
        pr_err("NW1728-driver: cdev_add failed: %d\n", result);
        goto err_unregister_chrdev;
    }

    g_class = class_create(class_name);
    if (IS_ERR(g_class)) {
        result = PTR_ERR(g_class);
        pr_err("NW1728-driver: class_create failed: %d\n", result);
        goto err_cdev_del;
    }

    g_device = device_create(g_class, NULL, g_devno, NULL, device_name);
    if (IS_ERR(g_device)) {
        result = PTR_ERR(g_device);
```

```
pr_err("NW1728-driver: device_create failed: %d\n", result);
goto err_class_destroy;
}

pr_notice("NW1728-driver: ready at /dev/%s (major=%d minor=%d)\n",
        device_name, MAJOR(g_devno), MINOR(g_devno));
return 0;

err_class_destroy:
class_destroy(g_class);
g_class = NULL;
err_cdev_del:
cdev_del(&g_cdev);
err_unregister_chrdev:
unregister_chrdev_region(g_devno, minor_count);
g_devno = 0;
err_free_gpio:
gpio_free(pin_rst);
err_free_cs:
gpio_free(pin_cs);
err_free_dc:
gpio_free(pin_dc);
err_free_din:
gpio_free(pin_din);
err_free_clk:
gpio_free(pin_clk);
err_out:
return result;
}

void unregister_device(void)
{
pr_notice("NW1728-driver: unregister_device() called\n");

if (!IS_ERR_OR_NULL(g_device)) {
device_destroy(g_class, g_devno);
g_device = NULL;
}
if (!IS_ERR_OR_NULL(g_class)) {
class_destroy(g_class);
g_class = NULL;
}
cdev_del(&g_cdev);
if (g_devno) {
unregister_chrdev_region(g_devno, 1);
g_devno = 0;
}
}
```

```
gpio_free(pin_rst);  
gpio_free(pin_cs);  
gpio_free(pin_dc);  
gpio_free(pin_din);  
gpio_free(pin_clk);  
pr_info("NW1728-driver: unregistered\n");  
}
```

Now, that is the SPI screen

5. Conclusions

Here is a summary of what I learned and achieved while building this custom character device driver:

- **Hardware Control and Bit-Banging:** I learned that you do not always need a built-in hardware controller to handle SPI communication. By using basic kernel functions like `gpio_request()` and `gpio_set_value()`, we can manually control generic pins to mimic an entire SPI protocol pipeline. It requires careful timing but gives us absolute flexibility.
- **Resource Safety and Clean Recovery:** Managing a kernel module forces you to be incredibly disciplined with memory and hardware management. Because ring 0 has no garbage collection, using `goto` statements to wind down allocations in strict reverse order is the only way to protect the system from memory leaks and fatal kernel panics during initialization failures.
- **User to Kernel Domain Separation:** Working with `copy_to_user()` and `copy_from_user()` showed me how strictly Linux isolates application memory from hardware memory. A driver must always serve as a secure gatekeeper, validating pointers and safely transferring blocks across the privilege wall before executing commands or serving strings.
- **Software and Hardware Synchronization:** Implementing the double-buffer setup (`lcd_text_buf` and `lcd_buf`) taught me how data undergoes multi-stage transformations. The driver catches raw text from a terminal, saves it in a tracking buffer so `cat` can read it back, translates it through a font character matrix, and pushes the final raw pixel stream to the physical LCD hardware.

And with that. the **RTOS Linux Driver Logbook** comes to and end. *Good Night!*